WNDRVR
SUPPORT NETWORK

VXWORKS 7 KERNEL APPLICATION AND RTP APPLICATION PROJECTS GUIDE, SR0650 EDITION 2

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Kernel Application and RTP Application Projects Guide, SR0650 Edition 2

17 August 2020

1. ABOUT KERNEL APPLICATION AND RTP APPLICATION PROJECTS

1. About Kernel Applications

VxWorks® kernel applications run in kernel mode as tasks, with full access to the system hardware.

During development, kernel modules are downloaded and dynamically linked to the operating system. This allows you to build, run, and debug applications without having to make any changes to the operating system itself. You can build modules separately, then run and debug them on a target running VxWorks. Modules can be loaded, unloaded, and reloaded as required.

Kernel application modules can be downloaded to a target from any host file system for which the kernel has support or can be stored on the target itself in flash or read-only memory, in a ROMFS file system, or on disk.

To deploy an application after development is complete, create a kernel module that is statically linked to the operating system, making it part of the VxWorks image itself. VxWorks can be configured to start kernel applications automatically at boot time.

A kernel application requires a downloadable kernel module (DKM) project to provide an environment for developing the application.

For information about writing kernel applications, see the *VxWorks 7 Application Programmer's Guide*.

Related information

Kernel Application Tutorials on page 5
Static Library Projects on page 15

2. About Real-Time Process Applications

A real-time process (RTP) application is a VxWorks executable that runs in user space as a process.

You can separately build, run, and debug the executable. At run-time, the executable file is loaded from the host or from a local VxWorks file system, and runs as an independent process. When the executable is loaded, the system allocates memory for it, execution begins at a known address, and all tasks in the process run within the same memory-protected space. Each process has its own memory space, that contains the following items:

- the executable program
- the program data
- stacks for each task
- the heap
- resources associated with the management of the process itself (such as memory-allocation tracking)

When the application terminates, all resources associated with it are released.

In addition to using RTP projects to create real-time process applications, you can also use them to create library subprojects that are statically linked into other projects at build time. You can make shared libraries accessible to your applications at run time, by configuring options for both the library and the application.

During development, you can store RTP applications on the host system and run them interactively. To deploy an application as a software product, you can store RTP applications in a target file system such as a ROMFS file system.

For information about writing RTP applications, see the *VxWorks 7 Application Programmer's Guide: RTP Applications*.

Related information

Real-Time Process Application Tutorials on page 7
Static Library Projects on page 15
About Shared Library Projects on page 21

3. About Static Libraries, Shared Libraries, and Plug-ins

Libraries can be included in kernel applications and real-time process (RTP) applications.

Static Libraries

A *static library* is a collection of precompiled functions bundled as an archive file with the **.a** extension and is integrated into kernel applications and RTP applications at compile time. Static libraries contain object code that is copied into the executable when your library is linked to the application at build time. A static library is useful if you want to include the same code in several downloadable or relinkable images, for example in different VIPs. However, each application in a system that references a static library includes a copy of the library in its execution code increasing the size of the software.

Shared Libraries

Shared libraries, also referred to as *dynamically-linked libraries*, or DLLs, contain functions and data, and are created for use in RTP applications at run-time. Shared libraries are dynamically linked to an application when the application is loaded. When an application is linked against a shared library at build time, the linker copies information about the name of the shared library (its shared object name) and its run-time location into the application. This information allows the dynamic linker to locate and load the shared library for the application automatically at run time. The shared library linker produces a *buildTargetName* **.so** target that is dynamically linked to at run time. The output product of the shared library linker is normally passed up to higher level projects (superprojects). Once loaded into memory by the dynamic linker, shared libraries are held in sections of memory (shared data areas) that are accessible to all applications. Each application that uses the shared library gets its own copy of the private data, which is stored in its own memory space. When the last application that references a shared library exits, the library is removed from memory.

Plug-ins

Plug-ins are similar to shared libraries, except that they are loaded on demand (programmatically) by the application instead of automatically at startup.

Both shared libraries and plug-ins are referred to generically as *dynamic shared objects* (DSOs). Applications that make use of shared libraries or plug-ins must be built as dynamic executables to include a dynamic linker in their image. The dynamic linker carries out the binding of the dynamic shared object and application at run time.

Static libraries and shared libraries perform essentially the same function. The key differences are as follows:

- Only the elements of a static library that are required by an application (that is, specific **.o** object files within the archive) are linked with the application. The entire library does not necessarily become part of the system. If multiple applications (*n* number) in a system use the same library elements, those elements are duplicated (*n* times) in the system—in both the storage media and system memory.
- The dynamic linker loads the entire shared library when any part of it is required by an application. As with a **.o** object file, a shared library **.so** file is an indivisible unit. If multiple applications in a system need the shared library, however, they share a single copy; the library code is not duplicated in the system.

For more information on using static libraries, shared libraries, and plug-ins, see the *VxWorks 7 Application Programmer's Guide: Static Libraries, Shared Libraries, and Plug-ins*.

Related information

Static Library Projects on page 15

WNDRVR

About Shared Library Projects on page 21
Build Requirements for Shared Libraries on page 21

4. VxWorks Projects and Tools

VxWorks projects enable the management of project resources for applications by collecting resources into logical and hierarchical structures.

Projects also enable the build of the same sources for different target architectures or different toolchains by changing the active build spec of the project (build properties).

Projects provide the infrastructure for kernel applications and real-time process (RTP) applications where you can add application code by importing existing resources or adding new resources.

You can add an application to a VxWorks image project (VIP) so that VxWorks starts the application automatically. Building a VIP with application subprojects helps assure correct linking and dependency checking. Downloadable kernel module (DKM) projects and real-time process (RTP) projects can be linked to a VIP this way, because they are solely source code containers. Linking object files or object codes to VxWorks images is done by editing the Makefile and setting the **EXTRA_MODULE** or **LIBS** macros.

VxWorks project tools include Workbench 4 and the wrtool command-line utility. The wrtool utility starts Workbench 4 without invoking the Workbench 4 graphical user interface.

For more information on Workbench 4, access the Wind River Knowledge Library and go to Workbench 4 documentation.

For more information on the wrtool utility, see the *VxWorks 7 Getting Started Guide*.

For information on how to create VxWorks source build (VSB) projects and VIPs, see the *VxWorks 7 Configuration and Build Guide*.

Related information

Downloadable Kernel Module Projects on page 5
Real-Time Process Projects on page 7
ROMFS File System Projects on page 10
Static Library Projects on page 15
About Shared Library Projects on page 21
Project Resource Management on page 39

5. Project Resources

Resources is a collective term for files, folders, and projects that exist in a project.

Files

Project files are comparable to files as you see them in a file system.

Folders

Project folders are comparable to directories on a file system and can contain files and other folders.

Projects

Projects contain files and may contain folders and other projects. When you create a project, you specify a location for it in the file system. A project is either open or closed. When a project is closed, it cannot be changed and is not examined during builds. When a project is open, you can access its contents and build the project.

Projects are used for builds, version management, sharing, and resource organization.

Within Workbench, project, folder, and file icons appear as follows:

Icon	Description
	Project (open)
	Folder
	File

Folders and files can be linked to a file system outside of the project's location. These special folders and files are called linked resources.

Project resources are organized in a hierarchy that includes a root, parent resources, and child resources.

Root

The top level in a file system.

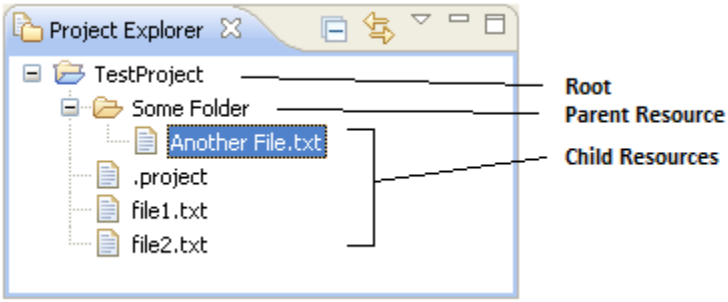
Parent Resource

A resource that contains another resource. Only projects and folders can be parent resources.

Child Resource

A resource that is contained within another resource. Only folders and files can be child resources.

Within Workbench 4, resource hierarchies are displayed in the Project Explorer view, which is one of the default views in the Resource perspective. For example:



Related information

Project Resource Management on page 39

2. DOWNLOADABLE KERNEL MODULE PROJECTS

1. Downloadable Kernel Module Projects

Downloadable kernel module (DKM) projects provide the development environment to manage and build kernel application modules that execute in kernel space.

You can also use DKM projects to create library subprojects that are statically linked into other projects at build time. For example, using the same code in multiple downloadable or relinkable images.

Workbench 4 allows you to base a DKM project on a VxWorks source build (VSB) project or a VxWorks image project (VIP). When using the the wrtool utility, you must base a DKM project on a VSB project.

For information on how to build VSB and VIP projects, see the *VxWorks 7 Configuration and Build Guide*.

Related information

Kernel Application Tutorials on page 5
Static Library Projects on page 15
Project Resource Management on page 39

2. Kernel Application Tutorials

The getting started tutorials show you how to create, build, and run kernel applications using Workbench 4 and the wrtool utility.

To create, build, and run CMake kernel applications use the same workflow.

The following kernel application tutorials are available:

- VxWorks 7 Kernel Application Tutorial - Workbench 4
  This tutorial provides instructions on how to create, build, and run a Hello World kernel application using Workbench 4.
- VxWorks 7 Kernel Application Tutorial - Command Line
  This tutorial provides instructions on how to create, build, and run a Hello World kernel application using the wrtool utility.

To access these tutorials from the Wind River Knowledge Library, go to *Getting Started* and select **Tutorials**.

To customize your CMake kernel applications, see: CMake Documentation.

3. Adding Partial Image Build Targets to DKM Subprojects

Subprojects appear as subnodes of a parent project, called a superproject. A subproject can pass build output to a superproject. If you include a DKM as a subproject of a VIP, the DKM must contain a partial image build target in the form of a **.o** file. If it does not contain a partial image the VIP will not build.

About This Task

The single, partially linked, not *munched* (not integrated with code to call C++ static constructors and destructors) **.o** file is for subproject support only. The **.o** file is not intended for download. When you include a DKM as a subproject, the partial image build

output is passed to the VIP superproject to be integrated into the downloadable module output, a munched **.out** file. For information on partially linked objects and munch, see the *VxWorks 7 Applications Programmer's Guide:* Kernel Applications.

You can check your DKM subproject build targets and add a partial image build target if it is not present.

Before You Begin

- Create a VSB based on a BSP.
- Create a VIP based on the VSB.
- Create a DKM based on the VIP and include the DKM in the VIP as a subproject.
  For more information on adding a project as a subproject, see Changing the Project Hierarchy.

Procedure

1. In the Project Explorer view of Workbench, expand the VIP superproject, the DKM subproject, and then **Build Targets**.

   If a **.o** build target is not present, proceed to the next step.

2. Right-click **Build Targets** and select **New Build Target**.
3. Enter a build target name.

   The build target name is repeated in the **Binary Output Name** field.

4. In the **Build tool** drop-down list, select **Partial Image Linker**.
5. Select the **Pass build target to the next level** check box.
6. Click **Finish**.

   The partial image build target appears in the build targets of the DKM subproject.

Results

The partial image build output will be passed to the VIP superproject when you build the VIP.

3. REAL-TIME PROCESS PROJECTS

1. Real-Time Process Projects

Real-time process (RTP) application projects provide the development environment to build executables that run as real-time processes in user mode (in memory protected user space).

Workbench 4 allows you to base an RTP project on a VxWorks source build (VSB) project or a VxWorks image project (VIP). When using the wrtool utility, you must base an RTP project on a VSB project.

For information on how to build VSB and VIP projects, see the *VxWorks 7 Configuration and Build Guide*.

Related information

Real-Time Process Application Tutorials on page 7
Static Library Projects on page 15
About Shared Library Projects on page 21
Project Resource Management on page 39

2. Real-Time Process Application Tutorials

The getting started tutorials show you how to create, build, and run real-time process (RTP) applications using Workbench 4 and the wrtool utility.

To create, build, and run CMake RTP applications use the same workflow.

The following RTP application tutorials are available:

- VxWorks 7 Real-Time Process Application Tutorial - Workbench 4
  This tutorial provides instructions on how to create, build, and run a Hello World RTP application using Workbench 4.
- VxWorks 7 Real-Time Process Application Tutorial - Command Line
  This tutorial provides instructions on how to create, build, and run a Hello World RTP application using the wrtool utility.

To access these tutorials from the Wind River Knowledge Library, go to *Getting Started* and select **Tutorials**.

To customize your CMake RTP applications, see: CMake Documentation.

4. ADDING BUILD TARGETS TO DKM AND RTP PROJECTS

When you create a project, Workbench displays a managed build project in the Project Explorer. The project contains the usual project-specific files, but you must create a build target manually.

Procedure

1. Expand your project, right-click **Build Targets**, and select **New Build Target**.

   The New Build Target dialog appears. By default, the Build target name and Binary output name are the same as the project name, but if you are going to create multiple build targets, give the build targets more descriptive names.

   Your build targets must have unique names, but you can use the same binary output name for each one. This allows you to deliver an output file with the same name in multiple configurations. Workbench adds a build tool-appropriate file extension to the name you type, so do not include the file extension in this field.

2. To use default content, choose the appropriate **Build tool** for your project; then click **Finish**.
3. Optionally, if you deselect **Use default content**, the **Next** button becomes selectable.

   If you click **Next**, the Content Definition dialog appears. It allows you to specify files, folders, projects or build targets to add.

4. Select **Show all projects** to display files, folders, and other build targets from outside your current project.

   If you have created a Working Set, you can restrict the display by selecting it from the pull-down list.

5. You can add contents to your build target in a number of ways.
   1. Select specific files, folders, projects, or other build targets in the left column and click **Add**.

      What you can add depends on the build tool you use. For example, you cannot add an executable build target to another build target.

      Folders or projects can be added flat or with recursive content.

      Using Add will create a flat structure, meaning that it adds the exact items you choose and skips any subfolders and files.

      2. Create a virtual folder within your build target by clicking **Add Virtual Folder**, typing a descriptive name in the dialog, and clicking **OK**.

      Virtual folders allow you to group objects within the build target so you can apply the same build settings to them; they also provide a way to add files with the same name from different locations.

6. To adjust the order of the build target contents, select items in the right column and click **Up**, **Down**, or **Remove**.

   Folders appear in the specified place in the list, but the files within them are added alphabetically.

7. When you have configured your build target, click **Finish**.

   It appears in the Project Explorer under the Build Targets node of your project.

WNDRVR

Note: Adding linked resources to a build target may cause problems within a team, if the linked resources are added using an absolute path instead of a variable. The variable must be maintained manually by the team or the location of the linked resource must be restricted.

Often, on Windows hosts, you might encounter command line length limitations when you are linking managed build targets that have many files. To avoid this limitation, you can use the capabilities of the toolchain to read command line options and arguments from a file. This helps to reduce the total number of characters on the command line.

WNDRVR

5. ROMFS FILE SYSTEM PROJECTS

1. ROMFS File System Projects

A ROMFS file system project provides the ability to bundle VxWorks applications and files with the operating system without the need for a local disk or a network connection to a remote disk.

When VxWorks is configured with the ROMFS file system, files of any type can be included in the operating system image by adding them to a ROMFS directory on the host system, and then rebuilding VxWorks. The build produces a single system image that includes both VxWorks and the files in the ROMFS file system directory.

When VxWorks is booted with this image, the ROMFS file system and the files it holds are loaded with the kernel. In addition, RTP applications can be coupled with an automated startup facility so that they run automatically at boot time. A ROMFS file system thereby provides the ability to create fully autonomous systems.

Related information

Configuring and Building a ROMFS File System Project on page 10
Configuring and Building a ROMFS File System Project without Workbench on page 12
Accessing Files in a ROMFS File System on page 13
Example Usage of ROMFS File Systems on page 14
Project Resource Management on page 39

2. Configuring and Building a ROMFS File System Project

A ROMFS file system project can be configured and built standalone or be part of a VxWorks source build (VSB) project, a VxWorks image project, or another project.

Before You Begin

You must have a VSB and VIP projects with ROMFS file system support.

Verify that the **FS** and **FS_ROMFS** options are enabled for the VSB project. If needed, rebuild the VSB project.

Verify that the **INCLUDE_ROMFS** component is included in the VIP. If needed, rebuild the VIP.

About This Task

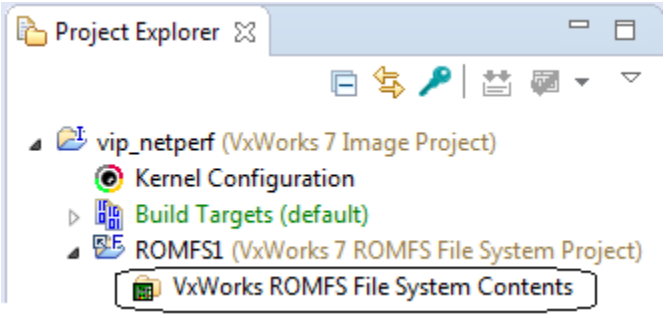
To configure and build a ROMFS file system for a VIP from Workbench 4, proceed as follows:

Procedure

1. Create a ROMFS file system project as a subproject to the VIP.
    1. Right-click the VIP from the Project Explorer and select **New > Project**.
    2. Select **VxWorks ROMFS File System Project** and click **Next**.
    3. Enter a name for the project and click **Finish**.

    The new ROMFS file system project appears in the Project Explorer under the VIP.

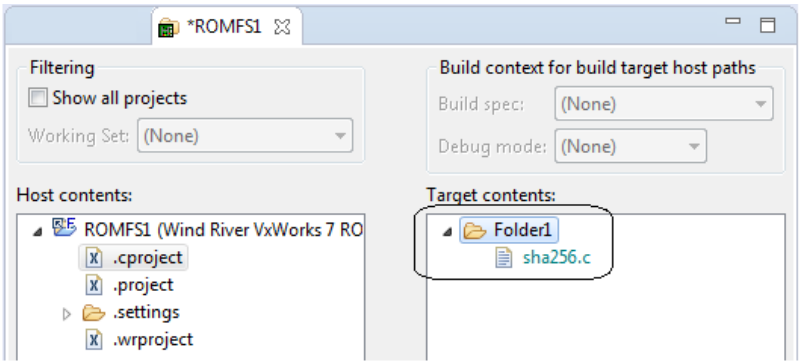
2. Open the newly created ROMFS file project by double-clicking the **VxWorks ROMFS File System Contents** item within the ROMFS File System subproject.



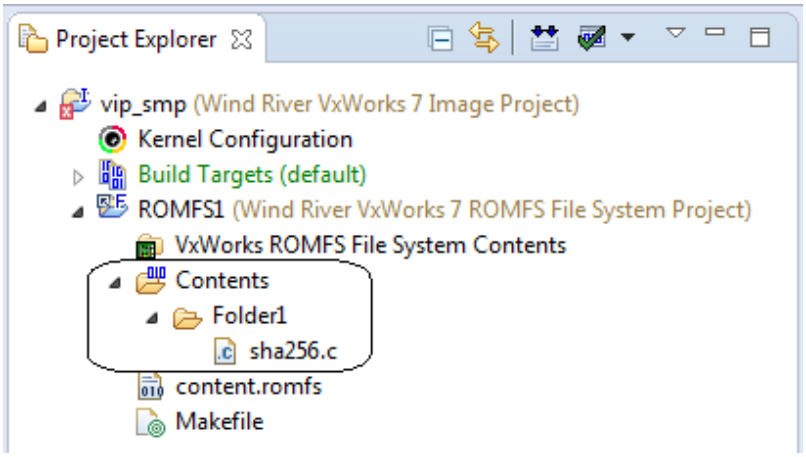
3. Add files and folders to the Target contents panel as required.
   1. To add a new folder, click **New Folder** and type a name for the folder.
   2. To add existing files and folders, click **Add External** and select files and folders.

> Note: From within the Target contents panel, you can drag and drop items in and out of folders to reorganize the ROMFS file system structure.

In this example, **Folder1** is added to the project and the **sha256.c** file is added to **Folder1**.



4. Save the ROMFS file system project.
5. Rebuild the VIP.
6. Observe the presence of the added items in a **Contents** folder within the ROMFS file system project.



Related information

ROMFS File System Projects on page 10
Configuring and Building a ROMFS File System Project without Workbench on page 12
Accessing Files in a ROMFS File System on page 13
Example Usage of ROMFS File Systems on page 14
Project Resource Management on page 39

3. Configuring and Building a ROMFS File System Project without Workbench

A ROMFS file system project can be configured and built to be part of a VxWorks image project (VIP) without being dependent on Workbench.

About This Task

The VIP build system includes a lightweight mechanism for adding a ROMFS drive to a VxWorks image. This is separate from the Workbench ROMFS File System Project and does not depend on Workbench. For more complex ROMFS content, the Workbench ROMFS File System is recommended.

Procedure

1. Create a VxWorks image project.
2. Add **INCLUDE_ROMFS** to the kernel configuration.
3. Create a ROMFS directory in the project directory.
4. Put the desired files and directories in the ROMFS directory.
5. Build the project.

Results

A ROMFS drive called **/romfs** with the contents of the ROMFS sub-directory is added to the VxWorks image.

Related information

ROMFS File System Projects on page 10
Accessing Files in a ROMFS File System on page 13
Example Usage of ROMFS File Systems on page 14
Configuring and Building a ROMFS File System Project on page 10
Project Resource Management on page 39

4. Accessing Files in a ROMFS File System

The content of the ROMFS file system directory can be browsed using the ls and cd shell commands, and accessed programmatically with standard file system functions, such as open( ) and read( ).

About This Task

To access a ROMFS file system from a target, do the following:

Note: For this example, the folder and file structure is as follows: **ROMFS1 > Contents > Folder1 > sha256.c**

Procedure

1. Boot your target.
2. From the target, access the command interpreter mode.

```
-> cmd
[VxWorks *]#
```

3. Access the folders and file using the cd and ls commands.

```
[VxWorks *]# cd/romfs
[VxWorks *]# ls
...
llvmcovHelloworld.out
llvmcovMerge_rtp.vxe
llvmcovMerge.out
ll
...
[VxWorks *]# pwd/romfs
[VxWorks *]# ls
...
llvmcovHelloworld.out
llvmcovMerge_rtp.vxe
llvmcovMerge.out
ll
...
[VxWorks *]# cd ll
[VxWorks *]# ls
...
llvmcovMerge_rtp.c
...
```

Related information

ROMFS File System Projects on page 10
Configuring and Building a ROMFS File System Project on page 10
Configuring and Building a ROMFS File System Project without Workbench on page 12
Example Usage of ROMFS File Systems on page 14

5. Example Usage of ROMFS File Systems

Example usage of ROMFS file systems are provided in a number of guides within the VxWorks 7 documentation.

For example usage of ROMFS file systems, see the following guides:

- VxWorks 7 CLI, Web, and MIBway Programmer's Guide

  As part of a Web server application, a ROMFS file system is used to store a **server.pem** certificate file.
- *Wind River RADIUS and Diameter for VxWorks 7 Programmer's Guide*

  As part of a security application with Transport Layer Security (TLS), a ROMFS file system is used to store a number of certificate and private key files.
- As part of the deployment phase of the example, a ROMFS file system is used to store the RTP application and libraries for the deployment of a software product.

Related information

ROMFS File System Projects on page 10
Configuring and Building a ROMFS File System Project on page 10
Configuring and Building a ROMFS File System Project without Workbench on page 12
Accessing Files in a ROMFS File System on page 13

6. STATIC LIBRARY PROJECTS

1. Static Library Projects

Static library projects enable the creation of a library of functions and data that can be statically linked to applications at build time.

A static library includes all of the object modules from the source. After building a static library, you can link it into other VxWorks image projects (VIPs) or application projects by passing its build target to the other project. When an application is linked against a static library at build time, the linker copies object code (**.o** files) from the library into the executable; they are statically linked.

The VxWorks development environment provides simple mechanisms for building static libraries (archives), including a useful set of default makefile rules.

There are two types of static library projects; static kernel library and static user library.

- A static kernel library project creates a library that can be statically linked to kernel applications at build time.
- A static user library project creates a library that can be statically linked to real-time process (RTP) applications at build time.

For more information on using static libraries, see the *VxWorks 7 Application Programmer's Guide: Static Library Development*.

Related information

About Static Libraries, Shared Libraries, and Plug-ins on page 2
Configuring and Building a Static Kernel Library Project on page 15
Configuring and Building a Static User Library Project on page 16
Creating a Static Library for VIP and DKM Projects on page 17
Linking a Static Library with a VIP on page 18
Linking a Static Kernel Library with a DKM Project on page 19
Project Resource Management on page 39

2. Configuring and Building a Static Kernel Library Project

Static kernel library projects provide libraries that can be statically linked to kernel applications at build time.

About This Task

To create, configure, and build a static kernel library project from Workbench 4, do the following:

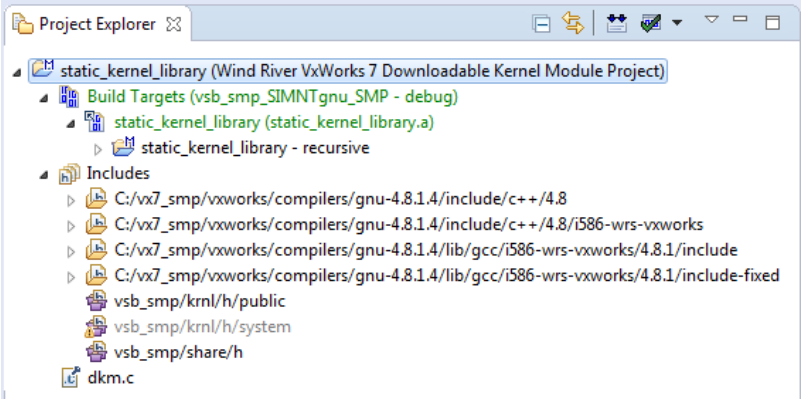
Procedure

1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
2. From the **Build type** drop-down list, select **Static Kernel Library**.
3. Click **Next**.
4. In the **Project name** text box, type the name of your project.
5. Click **Next**.
6. From the **Based on** drop-down list, select a project.
7. From the **Project** drop-down list, select a project, or **Browse** to your project.

For information on how to configure and build VSB and VIP projects, see the *VxWorks 7 Configuration and Build Guide*.

8. Click **Finish**.

   The new project appears in the Project Explorer. For example:



   Note: Workbench 4 automatically creates a stub C source file named **dkm.c** for the project.

9. Add the necessary resources to the project and build the project.

   See Importing Resources into Projects on page 40 and Adding New Resources to Projects on page 42.

Related information

About Static Libraries, Shared Libraries, and Plug-ins on page 2
Static Library Projects on page 15
Creating a Static Library for VIP and DKM Projects on page 17
Linking a Static Library with a VIP on page 18
Linking a Static Kernel Library with a DKM Project on page 19
Project Resource Management on page 39

3. Configuring and Building a Static User Library Project

Static user library projects provide libraries that can be statically linked to real-time process (RTP) applications at build time.

About This Task

To create, configure, and build a static user library project, do the following:

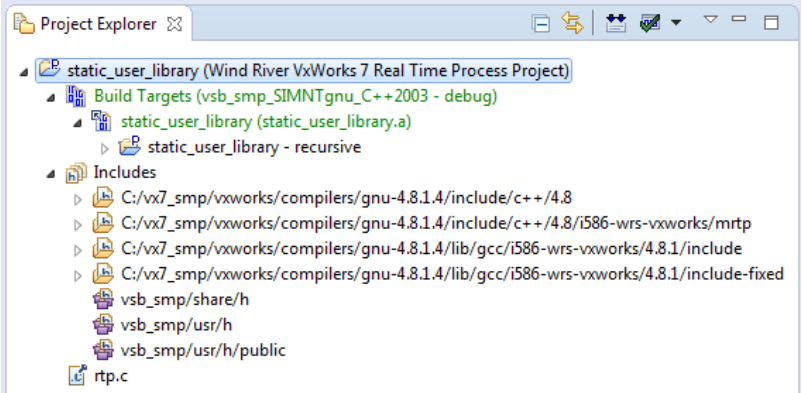
Procedure

1. Select **File > New > Wind River Workbench Project**.
2. From the dialogue window that appears, select an operating system.
3. From the New Wind River Workbench Project dialog, select **Static User Library**.
4. Following the instructions in the dialog, name the project and base the project on either a VxWorks source build (VSB) project or a VxWorks image project (VIP).

For information on how to configure and build VSB and VIP projects, see the *VxWorks 7 Configuration and Build Guide*.

5. Click **Finish**.

   The new project appears in the Project Explorer. For example:



   Note: Workbench 4 automatically creates a stub C source file named **rtp.c** for the project.

6. Add the necessary resources to the project and build the project.

   See Importing Resources into Projects on page 40 and Adding New Resources to Projects on page 42.

Related information

About Static Libraries, Shared Libraries, and Plug-ins on page 2
Static Library Projects on page 15
Creating a Static Library for VIP and DKM Projects on page 17
Linking a Static Library with a VIP on page 18

4. Creating a Static Library for VIP and DKM Projects

To create a static library that you can link to VxWorks image projects (VIPs) or downloadable kernel module (DKM) projects at build time, use a DKM project.

About This Task

The static library includes all of the object modules from the source. After building the static library, link it into other VIP or DKM projects by passing its build target to the projects.

Procedure

1. Create a DKM project.
   1. Select **File > New > Project**.
   2. Select **VxWorks Downloadable Kernel Module Project**.
   3. Name the project and click **Next**.
   4. Base the project on a VSB project or a VIP and click **Next**.

5. From the Build Target dialog, select **Librarian** from the **Build tool** drop-down list and select **Pass build target to next level**.
6. Click **Finish**.

2. Add source files to the DKM project.

   See Importing Resources into Projects on page 40 and Adding New Resources to Projects on page 42.

3. Build the project.

Related information

Configuring and Building a Static Kernel Library Project on page 15
Configuring and Building a Static User Library Project on page 16
Linking a Static Library with a VIP on page 18
Linking a Static Kernel Library with a DKM Project on page 19

5. Linking a Static Library with a VIP

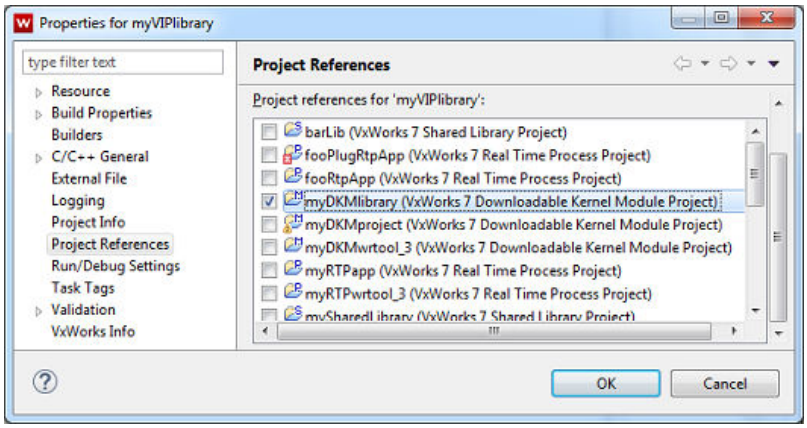
After creating and building a static library as part of a downloadable kernel module (DKM) project, you can link the project to a VxWorks image project (VIP) during the build.

Before You Begin

Before proceeding, build the static library specified in Creating a Static Library for VIP and DKM Projects on page 17.

Procedure

1. Create a VIP.
2. Add the static library as a sub project of the VIP.
   1. Right-click the VIP project in the Project Explorer, and select **Properties**.
   2. From the Properties dialog, select **Project References**.
   3. From the Project References pane, select the DKM library project. For example:



   4. Click **OK**.

3. From the Project Explorer, observe the presence of the library in the VIP project.
4. Build the VIP.

The build process links the DKM library project to the VIP project.

Related information

About Static Libraries, Shared Libraries, and Plug-ins on page 2
Static Library Projects on page 15
Configuring and Building a Static Kernel Library Project on page 15
Configuring and Building a Static User Library Project on page 16
Creating a Static Library for VIP and DKM Projects on page 17
Linking a Static Kernel Library with a DKM Project on page 19

6. Linking a Static Kernel Library with a DKM Project

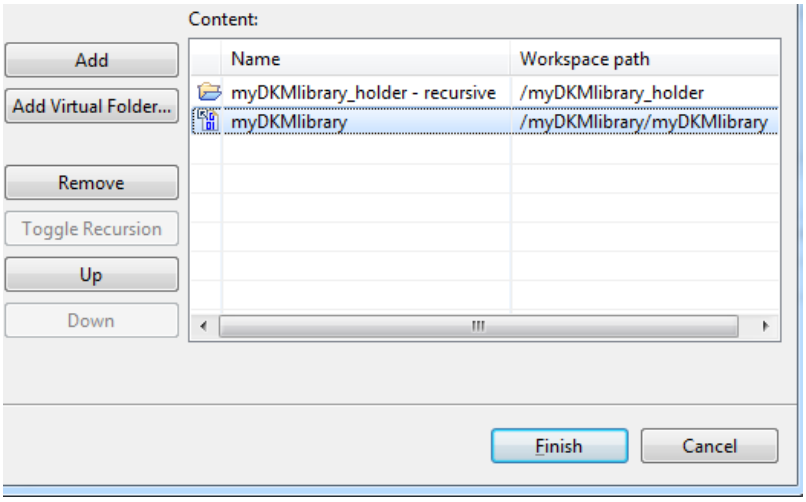
After creating and building a static library as part of a downloadable kernel module (DKM) project, you can link the project to another DKM project during the build.

Before You Begin

Before proceeding, build the static library specified in Creating a Static Library for VIP and DKM Projects on page 17.

Procedure

1. Create the DKM project you want to link the static library project to.
2. From the Project Explorer, right-click the partial image build target of the DKM project (*projectName* **_partialImage.o**) and select **Edit Content**.
3. From the Edit Content dialog, select **Show all projects** to display all available projects.
4. To add the library build target of the DKM library project to the project content, select the library (*libraryName* **.a**) and click **Add**.
5. Observe the presence of the library in the Content pane. For example:



6. Click **Finish**.
7. From the Project Explorer, observe the presence of the library in the DKM project.
8. Build the DKM project.

The build links the DKM library project to the new DKM project.

Related information

About Static Libraries, Shared Libraries, and Plug-ins on page 2
Static Library Projects on page 15
Configuring and Building a Static Kernel Library Project on page 15
Configuring and Building a Static User Library Project on page 16
Creating a Static Library for VIP and DKM Projects on page 17
Linking a Static Library with a VIP on page 18

WNDRVR

7. SHARED LIBRARY PROJECTS

1. About Shared Library Projects

Shared library projects are for libraries that are dynamically linked to real-time process (RTP) applications at run time.

Shared libraries can be stored on a host file system, a network file system, or a local file system on the target (including a ROMFS file system). You can also use shared library projects to create subprojects that are statically linked into other project types at build time.

Applications that make use of shared libraries must be built as dynamic executables to include a dynamic linker in their image. The dynamic linker carries out the binding of the dynamic shared object and application at run time.

Each shared library must be created with a shared object name, which functions as the run-time name of the library. The shared object name is used, together with other mechanisms, to locate the library at run time, and it can also be used to identify different versions of a library. By default, VxWorks uses the shared library file name for the shared object name.

2. Build Requirements for Shared Libraries

Shared library support in VxWorks is not provided by default, you must configure the VxWorks image project (VIP) and add kernel support for managing shared libraries.

You can simplify VIP configuration by using **BUNDLE_RTP_DEVELOP** and **BUNDLE_RTP_DEPLOY** bundle components.

Bundle Configuration

The **BUNDLE_RTP_DEVELOP** bundle provides support for shared libraries in a development system and includes the following components:

Component	Description
INCLUDE_RTP	The main component for real-time process support.
INCLUDE_RTP_APPL_USER	Enables the launch of the the application from a user-defined function
INCLUDE_RTP_SHOW	Adds show functions to enable the monitoring of RTPs.
INCLUDE_RTP_HOOKS	Enables the initialization of a shared library.
INCLUDE_SHARED_DATA	Enables the storage of shared library code.
INCLUDE_SHARED_DATA_SHOW	Adds show functions to display information on shared data regions.
INCLUDE_SHL	The main component for shared library support.
INCLUDE_SHL_SHOW	Adds show functions to display information about shared libraries in RTPs.

Component	Description
INCLUDE_RTP_SHELL_CMD	Adds shell commands for real-time processes.
INCLUDE_RTP_SHOW_SHELL_CMD	Adds command interpreter shell commands for real-time processes.
INCLUDE_SHL_SHELL_CMD	Adds command interpreter shell commands for shared libraries.

The **BUNDLE_RTP_DEPLOY** bundle provides support for shared libraries in a deployed system and includes the following components:

- **INCLUDE_RTP**
- **INCLUDE_RTP_APPL_USER**
- **INCLUDE_RTP_HOOKS**
- **INCLUDE_SHARED_DATA**
- **INCLUDE_SHL**

Minimum Configuration

For a minimum VIP configuration for shared libraries, include the following components:

- **INCLUDE_SHL**
- **INCLUDE_RTP**
- **INCLUDE_SHARED_DATA**
- **INCLUDE_RTP_HOOKS**

Related information

About Static Libraries, Shared Libraries, and Plug-ins on page 2
About Shared Library Projects on page 21
Configuring and Building a Shared Library Project on page 22

3. Configuring and Building a Shared Library Project

Shared library projects provide libraries that can be dynamically linked to real-time process (RTP) applications at run time.

About This Task

To create, configure, and build a shared library project, do the following:

Procedure

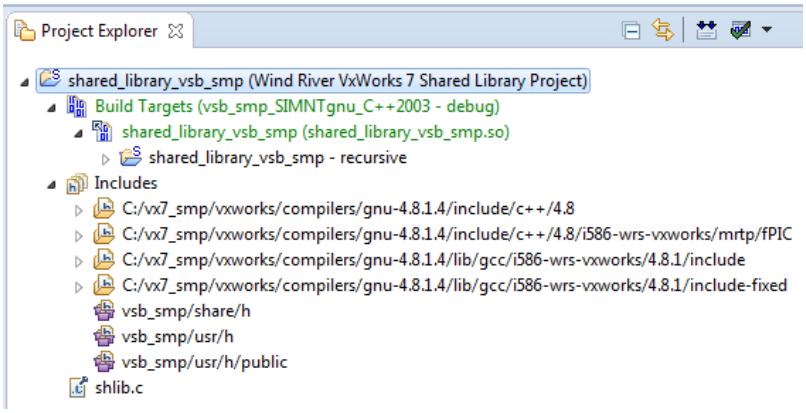
1. Select **File > New > Wind River Workbench Project**.
2. From the dialogue window that appears, select an operating system.
3. From the dialog, select **Shared User Library**. Click **Next**.

4. Following the instructions in the dialog, name the project, and base the project on a VxWorks source build (VSB) project or a VxWorks image project (VIP).

   For information on how to configure and build VSB and VIP projects, see the *VxWorks 7 Configuration and Build Guide*.

5. Click **Finish**.

   The new project appears in the Project Explorer. For example:



> **Note:** The default name for a shared library is based on the project name; the syntax is *projectName* **.so**.
>
> For this project, the shared library name is **shared_library_vsb_smp.so**.
>
> Workbench automatically creates a stub C source file named **shlib.c** for the project.

6. Add the necessary resources to the project.

   See Importing Resources into Projects on page 40 and Adding New Resources to Projects on page 42.

7. Build the project.

Related information

About Static Libraries, Shared Libraries, and Plug-ins on page 2
About Shared Library Projects on page 21
Build Requirements for Shared Libraries on page 21

4. Shared Library Location and Loading at Run-time

The dynamic linker must be able to locate a shared library at run-time. It uses the shared object name stored in the dynamic application to identify the shared library file, but it also needs information about the location of the file (on the target system, host, or network).

There are a variety of mechanisms for identifying the location of shared libraries at run-time. In addition, the dynamic linker can be instructed to load a set of libraries at startup time, rather than when the application that needs them is loaded (that is, to *preload* the shared libraries).

Specifying Shared Library Locations: Options and Search Order

In conjunction with shared libraries' shared object names (in ELF **SONAME** records), the dynamic linker can use environment variables, configuration files, compiled location information, and a default location to find the shared libraries required by its applications.

After determining that required libraries have not already been loaded, the dynamic linker searches for them in the directories identified with the following mechanisms in the order listed:

1. The VxWorks environment variable **LD_LIBRARY_PATH**.
2. The configuration file **ld.so.conf**.
3. The ELF **RPATH** record in the application's executable (created with the -rpath linker option).
4. The same directory as the one in which the application file is located.

In addition, the VxWorks **LD_PRELOAD** environment variable can be used to identify a set of libraries to load at startup time, before loading any other shared libraries.

Note that the dynamic linker loads shared libraries in the order in which it encounters **NEEDED** records. It executes the constructors in each shared library in reverse order of loading.

Using the LD_LIBRARY_PATH Environment Variable

The **LD_LIBRARY_PATH** environment variable can be used to specify shared library locations when an application is started. If set, it is the first mechanism used by the dynamic linker for locating libraries. The **LD_LIBRARY_PATH** environment variable is useful as an alternative to the other mechanisms, or as a means to override them.

Note: If the file is not stored on the target system, the path must be valid from the point of view of the target itself.

Note: For Windows hosts you must use forward slashes (or double backslashes) as path delimiters. This is the case even when the executable is stored on the host system.

Setting LD_LIBRARY_PATH From the Shell

Using the shell's command interpreter, for example, the syntax for using **LD_LIBRARY_PATH** (in this case within the rtp exec command) would be as follows:

```
rtp exec -e "LD_LIBRARY_PATH=libPath1 ;libPath2 " exePathAndName arg1 arg2 . . .
```

Note in particular that there are no spaces within the quote-enclosed string, that the paths are identified as a semicolon-separated list; and that the one space between the string and the executable argument to rtp exec is optional (the shell parser removes spaces between arguments in command-line mode).

If, for example, the shared libraries were stored in ROMFS on the target in the **lib** subdirectory, the command would look quite tidy:

```
[vxWorks *]#rtp exec -e "LD_LIBRARY_PATH=/romfs/lib" /romfs/app/myVxApp.vxe one two three
```

The command for shared libraries stored in an NFS directory would look very similar, with the path starting with the mount point on VxWorks.

Setting LD_LIBRARY_PATH From Application Code

The following code fragment illustrates setting **LD_LIBRARY_PATH** from application code, which would be necessarily be done before any RTP application that required the specified library was spawned:

```
#include <envLib.h>
...
putenv("LD_LIBRARY_PATH=/nfsdir/foo/libs");
```

Using the ld.so.conf Configuration File

The second-priority mechanism for identifying the location of shared libraries is the configuration file **ld.so.conf**.

The **ld.so.conf** file simply lists paths, one per line. The pound sign (#) can be used at the left margin for comment lines. By default the dynamic linker looks for **ld.so.conf** in the same directory as the one in which the application executable is located. The location of **ld.so.conf** can also specified with the VxWorks **LD_SO_CONF** environment variable.

Using the ELF RPATH Record

The third-priority mechanism for identifying the location of shared libraries is the ELF **RPATH** record in the application's executable.

The ELF **RPATH** record is created if an application is built with the -rpath linker option to identify the run-time locations of shared libraries. For example:

```
-rpath /romfs/lib
```

Here, the **lib** subdirectory in the ROMFS file system is defined as the location for shared libraries.

Using the Application Directory

If none of the other mechanisms for locating a shared library have worked, the dynamic linker checks the directory in which the application itself is located.

By default, the VxWorks makefile system creates both the application executable and run-time shared library files in the same directory, which facilitates running the application during the development process. Workbench manages projects differently, and creates applications and shared libraries in different directories.

Pre-loading Shared Libraries

By default, shared libraries are loaded when an application that makes use of them is loaded. There are two mechanisms that can be used to load libraries in advance of the normal loading procedure: using the **LD_PRELOAD** environment variable and using a dummy RTP application.

Pre-Loading Shared Libraries with LD_PRELOAD

The VxWorks **LD_PRELOAD** environment variable can be used to identify a set of libraries to load at startup time, before any other shared libraries are loaded (that is, to *preload* them). The variable can be set to a semicolon-separated list of absolute paths to the shared libraries that are to be pre-loaded. For example:

```
/romfs/lib/libMyFoo.so.1;c:/proj/lib/libMyBar.so.1;/home/moimoi/proj/libMoreStuff.so.1
```

A typical use of pre-loaded shared libraries is to override definitions in shared libraries that are loaded in the normal manner. The definition of symbols by pre-loaded libraries take precedence over symbols defined by an application or any other shared library.

Note that even pre-loaded shared libraries are removed from memory immediately if no RTP application makes use of them. To keep them in memory the dummy RTP method must be used (for information in this regard, see below).

Pre-Loading Shared Libraries with a Dummy RTP Application

Startup speed can be increased by using a dummy RTP to load all required libraries in advance, so that the other RTP applications spend less time at startup. The dummy RTP should be built to require all the necessary libraries, should be designed to remain in the system at least until the last user of the loaded libraries has started. The reference counting mechanism in the kernel ensures that the libraries are not unloaded.

WNDRVR

8. EXAMPLE IMPLEMENTATION OF A SHARED LIBRARY

1. Overview of the Shared Library Implementation

This procedure creates a real-time process (RTP) application that automatically calls a dynamically linked application from a shared library at start up.

About this Example

This procedure is divided into five major tasks designed to be followed sequentially. It is a simple example based upon a previously built VSB with RTP, dynamic linker and ROMFS support. In this example, the VSB is built upon the **vxsim_windows** simulator BSP, but other BSPs can be substituted.

Four projects are created and configured: a real-time process (RTP) project, a shared library project and a VxWorks image project (VIP) with a ROMFS sub-project. To do this example perform the following tasks in the order that follows:

1. Creating the Shared Library Project on page 27
   You create a shared library project based upon a properly configured BSP. A header file is added to the project containing bar ( ). A source file (**barLib.c**) is added with a simple printf command to confirm when the bar ( ) function is called by the application.
2. Creating the RTP Application on page 29
   You create a RTP project that contains an application called **fooRtpApp.vxe**. The RTP contains a source file (**fooRtpApp.c**) that, like the shared library project, contains a simple printf command to confirm when it executes.
3. Creating the VxWorks Image Project on page 33
   You create a VxWorks kernel image project (VIP) that contains a ROMFS file system subproject. The ROMFS is configured to store the application and libraries
4. Connecting to the Target and Running the Example RTP Application on page 34
   You connect to a target and manually launch the RTP application to verify that it executes properly.
5. Deploying the RTP Application on page 36
   Lastly, you rebuild the RTP project as a dynamic executable containing the RTP application, ROMFS file system and the shared libraries, then confirm that the process works automatically upon startup.

Next topic: Creating the Shared Library Project on page 27

2. Creating the Shared Library Project

This is the first task in the Example Implementation of a Shared Library procedure.

About This Task

You create a shared library project based upon a properly configured existing VSB. A header file is added to the project containing a function called bar( ). A source file (**barLib.c**) is added with a simple printf command to confirm when the bar( ) function is called by the application.

Before You Begin

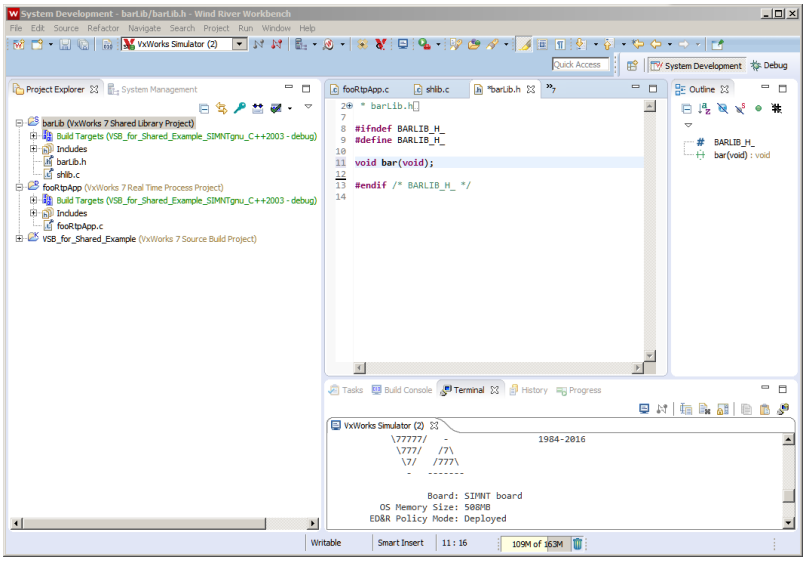
You should have, or performed the following:

- Read Overview of the Shared Library Implementation on page 27.
- General knowledge of VxWorks 7 shared libraries. See About Shared Library Projects on page 21.
- An existing VSB with a RTP, dynamic linker and ROMFS support configured.

Procedure

1. Select **File > New > Project**.
2. Select an operating system from the dialog window that appears.
3. Select **VxWorks 7 > VxWorks Shared Library Project** and click **Next**.
4. Type **barLib** as the project name, then click **Next**.
5. In the **Based on** drop-down list, select **a source build project** and in the **Project** drop-down list select the VSB of your choosing and click **Next**.
6. Ensure that **Shared Library Linker** is selected in the **Build Tool** drop-down list and click **Finish**.
7. Right-click the **barLib** project in Project Explorer and select **New > Header File**.
8. In the **Header file** text box, type **barLib.h** and select Default C header template from the drop-down menu for **Template**. Then click **Finish**.
9. In the **barLib.h** tab of the editor window, add the following code into the **barLib.h** file and save the file:

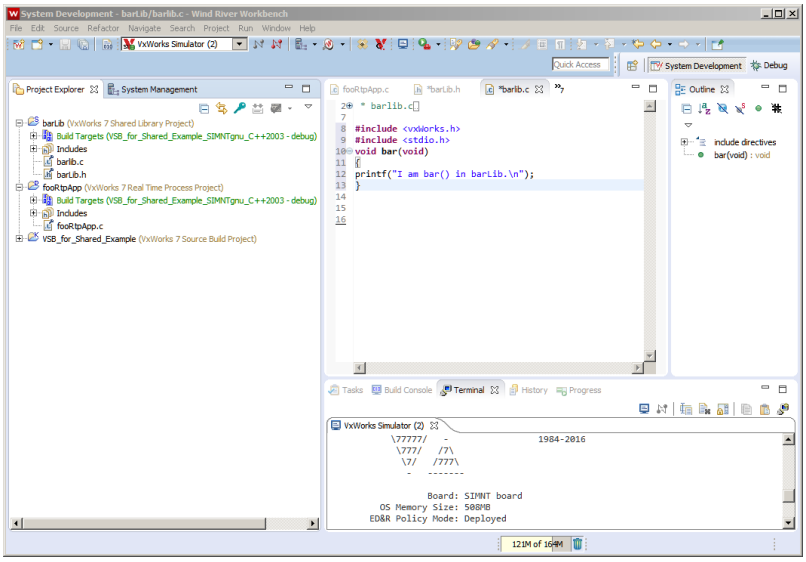
```
void bar(void);
```



10. Right-click the **barLib** project again and select **New > Source File**.
11. In the **Source file** text box, type **barLib.c** and select Default C source template from the drop-down menu for **Template**. Then click **Finish**.
12. **Right-click** the unneeded **shlib.c** file and **Delete** it from the **barLib** project.
13. In the **barLib.c** file editor window, add and save the following code in the **barLib.c** file:

```
#include <vxWorks.h>
#include <stdio.h>
void bar(void)
{
```

```
printf("I am bar() in barLib.\n");
}
```



Previous topic: Overview of the Shared Library Implementation on page 27
Next topic: Creating the RTP Application on page 29

3. Creating the RTP Application

This is the second task in the Example Implementation of a Shared Library procedure.

About This Task

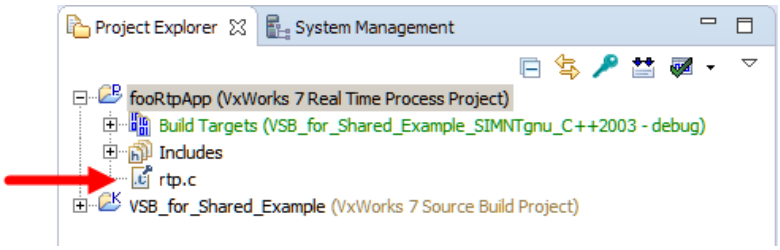
In this task you create the RTP application based upon the VSB used in the previous task Creating the Shared Library Project on page 27 and add the application's source file.

Before You Begin

You should have completed Creating the Shared Library Project on page 27.

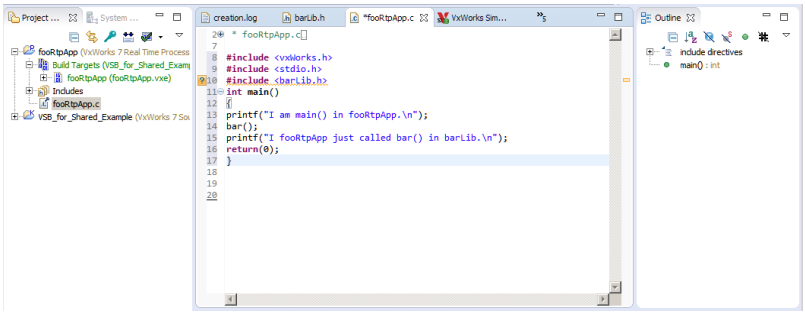
Procedure

1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
2. In the **Build Type** drop-down list, select **Real Time Process** and click **Next**.
3. In the **Project name** text box type **fooRtpApp** and click **Next**.
4. In the **Based on** drop-down list ensure **a source build project** is selected and in the **Project** drop-down list select a VSB project of your choosing and click **Finish**.
5. In the Project Explorer expand the **fooRtpApp** project and delete the unneeded **rtp.c** file.



6. In the Project Explorer right-click the **fooRtpApp** project, and select **New > Source File**.
7. Type **fooRtpApp.c** in the **Source file** text box and click **Finish**.
8. Add the following code into the **fooRtpApp.c** file and save the file.

```
#include <vxWorks.h>
#include <stdio.h>
#include <barLib.h>
int main()
{
printf("I am main() in fooRtpApp.\n");
bar();
printf("I fooRtpApp just called bar() in barLib.\n");
return(0);
}
```



Previous topic: Creating the Shared Library Project on page 27
Next topic: Setting Up a Build Target for fooRtpApp on page 30

4. Setting Up a Build Target for fooRtpApp

This is the third task in the Example Implementation of a Shared Library procedure.

About This Task

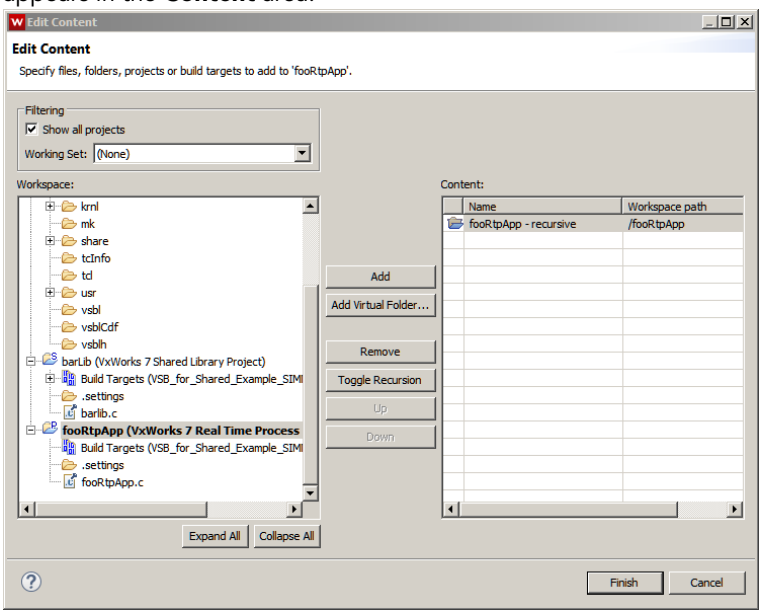
Setting up the build-time linkage of an application, such as **fooRtpApp**, that uses a shared library creates an executable and linkable format record (ELF) in the application with the shared object name of the shared library. The dynamic linker uses this name at run time to load the shared library when the application is loaded.

Before You Begin

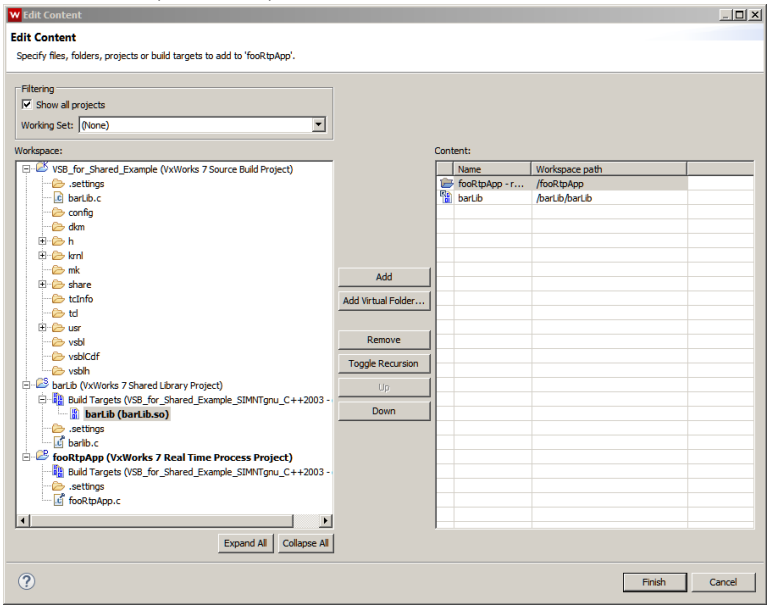
You should have completed Creating the RTP Application on page 29.

Procedure

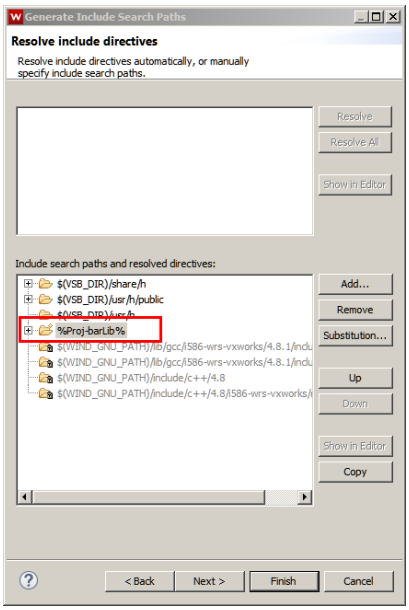
1. Expand **Build Targets** and right-click on **fooRtpApp (fooRtpApp.vxe)**, select **Edit Content** and verify that **fooRtpApp - recursive** appears in the **Content** area.



2. Click the **Show all projects** checkbox and in the **Workspace** area expand the contents of the **barLib** project until **barLib.so (barLib.so)** appears.
3. Select **barLib (barLib.so)** and click **Add** to add the item to the **Content** area then click **Finish**



4. Right-click the **fooRtpApp** project and select **Build > Generate Include Search Paths**. Ensure in the **Analyze include directives** window that the **Ignore all system directives** checkbox is unchecked and click **Next**.
5. In the **Resolve include directives** window, click **Resolve All**. Ensure that **%Proj-barLib%** is added into the **Include search paths and resolved directories** (see figure below) and click **Finish**. Rebuild the index when prompted to do so.



6. Right-click **fooRtpApp (fooRtpApp.vxe)** and select **Properties**, select Build Properties and in the **Tools** tab, verify that **Build tool** drop-down list is set to **Linker**.
7. Click **Tools Flags** and ensure that the **Create dynamic executable** checkbox is selected. Click **OK** then click the **Apply and Close** button to close the Build Properties window.

   In the **Tools Flags**, no **-static** appears for the LLVM compiler.

8. If your libraries have dependencies on other libraries (including system libraries like **libc++**, **libc**, etc), make sure you add the user libraries first and then add any other system libraries required to the **LIBS** macro. The link order is important.
9. Right-click and select **Build Project** for both **barLib** and **fooRtpApp** projects. If a **Build** dialog appears, click **Continue** to generate the include paths. If prompted, click Yes to rebuild the index.

   Verifying the Contents of the fooRtpApp Executable on page 32 This is the fourth task in the Example Implementation of a Shared Library procedure.

Previous topic: Creating the RTP Application on page 29
Next topic: Creating the VxWorks Image Project on page 33

4.1. Verifying the Contents of the fooRtpApp Executable

This is the fourth task in the Example Implementation of a Shared Library procedure.

About This Task

These steps verify that the appropriate records have been created. Use the readelf command, appending the appropriate architecture name (either arm, pentium, or ppc) along with the -d option to ensure that the correct ELF NEEDED records are created. For example, *farm* .

The VxWorks distribution provides the **libc.so** C run-time shared library. The **libc.so.1** library provides all of the basic facilities that an application requires.

The readelf utility extracts records from the GNU compiler outputs for a given architecture. To get help for the readelf utility, use the readelf*archName* -H command.

Before You Begin

You should have completed Setting Up a Build Target for fooRtpApp on page 30.

Procedure

1. Select **Project > Open Development Shell > VxWorks 7 > OK**.
2. In the VxWorks 7 **Terminal** window, execute the readelf command, appending your particular architecture (either arm, pentium, or ppc) while including the -d option.

   The syntax is as follows:

   readelf*architectureName* -d *absoluteFilePath/***fooRtpApp.vxe**

   Confirm the RTP is properly configured by ensuring that both **barLib.so**, **libc.so.1**, and **libllvm.so.1** are listed as NEEDED as shown in the following example:

```
C:\vxworks_1\workspace>readelfpentium -d C:\vxworks_1\workspace\fooRtpApp\VSB_for_Shared_librar
y_SIMNTgnu_C++2003\fooRtpApp\Debug\fooRtpApp.vxe

Dynamic section at offset 0x10020 contains 20 entries:
  Tag        Type                         Name/Value
 0x00000001 (NEEDED)                     Shared library: [barlib.so]
 0x00000001 (NEEDED)                     Shared library: [libc.so.1]
 0x00000001 (NEEDED)                     Shared library: [libllvm.so.1]
```

Parent topic: Setting Up a Build Target for fooRtpApp on page 30

5. Creating the VxWorks Image Project

This is the fifth task in the Example Implementation of a Shared Library procedure.

About This Task

The following procedure creates a VIP and adds the necessary components to support real-time processes (RTPs). You create a VxWorks kernel image project (VIP) that contains a ROMFS file system subproject. The ROMFS is configured to store the application and libraries.

Before You Begin

You should have completed Creating the RTP Application on page 29.

Procedure

1. Select **File > New > Wind River Workbench Project** . In the **Build Type** drop-down list, select **VxWorks Image Project** and click **Next**.
2. Enter a project name and click **Next**.

3. In the **Based on** drop-down list, select **a source build project**. In the **Project** drop-down list ensure the VSB you previously created is selected and click **Next**.
4. From the **Profile** drop-down list, select **PROFILE_DEVELOPMENT** and click **Finish**.
5. If necessary, add the **Real-Time Process Development** (**BUNDLE_RTP_DEVELOP**) bundle to the VIP (they are included if using the simulator). In the Project explorer expand the VIP, then right click the **Kernel Configuration** and select **Edit Kernel Configuration**. Select the **Bundles** tab to determine if the bundle is installed.
6. If necessary, add support for your hardware target under the **FOLDER_HARDWARE** component (already included if using the simulator)). Edit the kernel configuration, select the **Components** tab and search **FOLDER_HARDWARE** to determine if the component is installed.
7. Save the VIP configuration.
8. Right-click on the VIP and select **Build Project**.

Previous topic: Setting Up a Build Target for fooRtpApp on page 30
Next topic: Connecting to the Target and Running the Example RTP Application on page 34

6. Connecting to the Target and Running the Example RTP Application

This is the sixth task in the Example Implementation of a Shared Library procedure.

About This Task

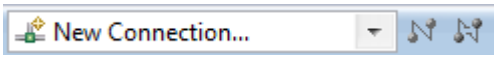
In this task you load the kernel image onto the target and verify that the RTP executable runs correctly.

Before You Begin

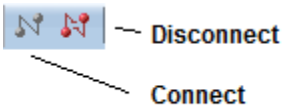
You should have completed Creating the VxWorks Image Project on page 33.

Procedure

1. From the Workbench tool bar, select **New Connection** from the drop-down list and enter a **Connection Name**.



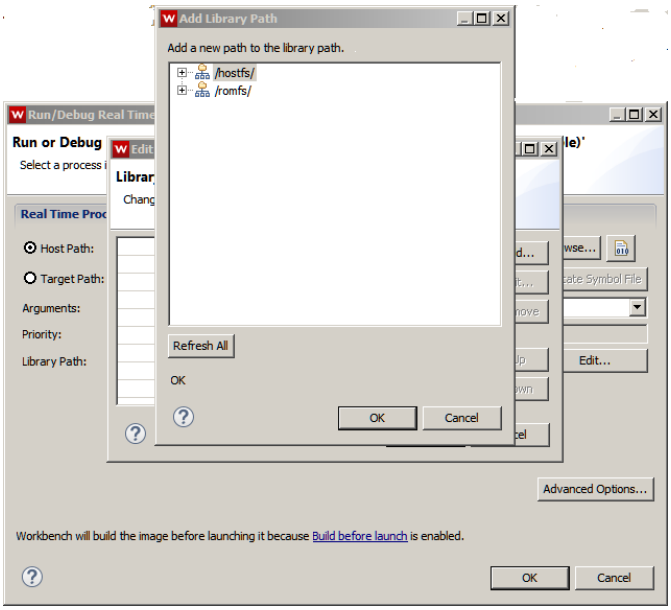
2. In the Target Type area, select **VxWorks Simulator** (or **Running Target** if you are not using the simulator).
   1. In the **Connection Mode** area select **Application Mode**.
   2. From the **Kernel Image** drop-down list, select the path to the kernel image created in the workspace, or if necessary **Browse** to navigate to the kernel image and click **Finish**.
3. Click the connect button to connect to the target if it did not connect automatically.

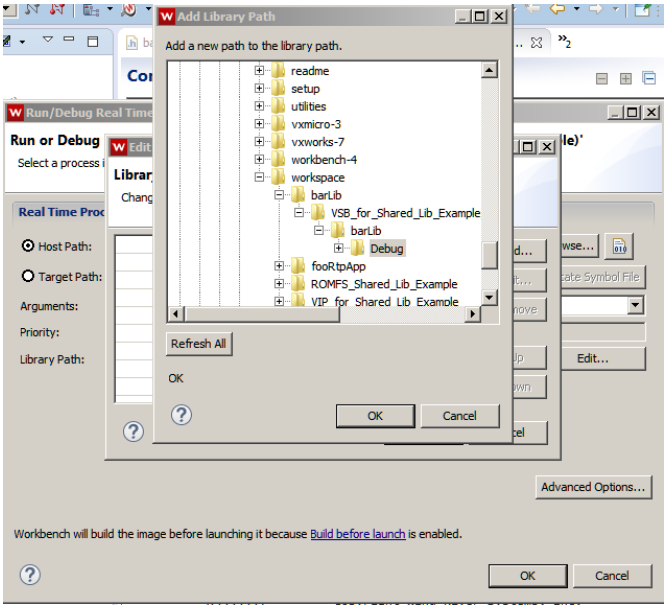


4. Right-click the **fooRtpApp** project and select **Run/Debug Real Time Process**. Ensure the Host Path drop-down list has the path to **fooRtpApp.vxe**Creating the RTP Application on page 29.
5. Click the **Library Path, Edit** button.
6. In the **Add Library Path** window, click **Add** and navigate through the **/hostfs/** tree to find the **barLib.so** file. The path to the file appears differently than using the Windows Explorer (see Info below). Do **not** click OK.

   The **barLib.so** file is in the /hostfs/ Debug folder in the **barlib** project.

WNDRVR





7. Click **Add** again and find the **libc.so.1** file in a similar fashion as was done for the **barLib.so** file and click **OK** to save the paths and OK again to exit the **Run or Debug Real Time Process** window.

   The **libc.so.1** file is in the bin folder in the **vsb/usr/root/llvm** project.

8. After clicking **OK** from the **Run or Debug Real Time Process** window, the RTP executes the **fooRtpApp** application which calls the **barLib** library and displays the following confirming proper operation:

```
I am main() in fooRtpApp.
I am bar() in barLib.
I fooRtpApp just called bar() in barLib.
```

Previous topic: Creating the VxWorks Image Project on page 33
Next topic: Deploying the RTP Application on page 36

7. Deploying the RTP Application

This is the seventh and final task in the Example Implementation of a Shared Library procedure.

About This Task

In this final phase of the example, you statically identify the run-time location of the shared library instead of typing the entire **LD_LIBRARY_PATH** command when launching the application. You add initialization code, so that VxWorks can start the application automatically.

Before You Begin

You should have completed Connecting to the Target and Running the Example RTP Application on page 34.

Procedure

1. In **fooRtpApp**, right-click the build target **fooRtpApp** (**fooRtpApp.vxe**), then select **Properties**.
2. In the Build Properties dialog, select the Tools tab and verify that the **Build tool** drop-down list is set to **Linker**.
3. Click Tool Flags, verify that in the **Options Create a dynamic executable** is selected, then click **OK**.
4. In the Tool Flags text field, add the options that define the location of the shared library files on your target (known as the *run path*), and then click OK.

   Use either of the following syntax examples:

   ```
   -rpath/romfs/lib
   ```

   ```
   -Wl,-rpath,/romfs/lib
   ```

   The **/romfs** directory is the default root directory used in the ROMFS file system project that you will create later in this procedure.

5. Right-click the **fooRtpApp** project and select **Rebuild Project**.
6. Right-click the VIP project and select **New > Project**.
7. Select **VxWorks ROMFS File System Project**, then click **Next** to create the ROMFS file system project in the VIP.
8. In the Project Explorer expand the VIP project, right-click on the newly created ROMFS project and select **Edit Contents**.
9. Click **New Folder** and create a folder named **bin**. Then click **New Folder** again to create another folder named **lib**.
10. Click **bin** then click **Add External** and navigate to the **fooRtpApp.vxe** file. Double-click **fooRtpApp.vxe** to add it into the **bin** folder.

    The fooRtpApp.vxe file is located in the VSB **fooRtpApp/Debug** folder.

11. Repeat step 10 inserting the **barLib.so**, **libc.so.1**, and **libllvm.so.1** files into the **lib** folder. Example locations for these files are as follows:

    barLib.so is located in *installDir* **/workbench-4/workspace/***vsbProjectName* **/barlib/Debug/***compilerName*

    libc.so.1 is located in *installDir* **/workbench-4/workspace/***vsbProjectName* **/usr/root/***compilerName* **/bin/**

    libllvm.so.1 is located in *installDir* **/workbench-4/workspace/***vsbProjectName* **/usr/root/***compilerName* **/bin/**

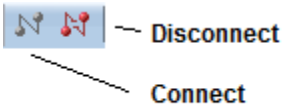
    ```
    Example paths:
    ```


WNDRVR

```
C:/vxworks_1/workspace/barLib/VSB_for_Shared_Lib_Example_SIMNTgnu_C++2003/barLib/Debug/barLib.s
o
```

```
C:/vxworks_1/workspace/VSB_for_Shared_Lib_Example/usr/root/llvm/bin/libc.so.1
```

12. Save the configuration and perform a **Rebuild Project** on the VIP.
13. From the Workbench task bar, select the Connection (if necessary) then click **Connect**.



14. Access the command shell and display the file system contents from the Terminal tab. The following output confirms the proper locations of the library files and application:

```
-> cmd
[vxWorks *]# ls "/romfs/bin"
/romfs/bin/.
/romfs/bin/..
/romfs/bin/fooRtpApp.vxe

[vxWorks *]# ls "/romfs/lib"
/romfs/lib/.
/romfs/lib/..
/romfs/lib/libc.so.1
/romfs/lib/barLib.so

[vxWorks *]#
```

15. From the Terminal, add **LD_LIBRARY_PATH** using the following command:

```
[vxWorks *]# rtp exec -e "LD_LIBRARY_PATH=romfs/lib" "romfs/bin/fooRtpApp.vxe"
```

16. From the Terminal, launch the **fooRtpApp** application using the following command:

```
[vxWorks *]# rtp exec "/romfs/bin/fooRtpApp.vxe"
```

The following output shows that the RTP application launches and calls the shared library:

```
[vxWorks *]# rtp exec "/romfs/bin/fooRtpApp.vxe"

Launching process '/romfs/bin/fooRtpApp.vxe' ...
Process '/romfs/bin/fooRtpApp.vxe' (process Id = 0x11739178) launched.
I am main() in fooRtpApp.
I am bar() in barLib.
I fooRtpApp just called bar() in barLib.
```

17. Click **Disconnect** to disconnect from the target.
18. From the VIP project in the Project Explorer, double-click **userRtpAppInit.c** to open the file for editing.
19. Add the following lines of code to the usrRtpAppInit( ) function to start **fooRtpApp.vxe** automatically. For more information on starting VxWorks applications automatically, see the *VxWorks 7 Application Programmer's Guide: Using userAppInit() to Run Applications Automatically*.

```
/* Includes */
#include <vxWorks.h>
```

```
#include <rtpLib.h>
#include <stdio.h>
#include <errno.h>
/*********************************************************************
*
* usrRtpAppInit - initialize the user's RTP application.
*/
void usrRtpAppInit (void)
{
 /* add your own application launch code here */
 char * vxeName = "/romfs/bin/fooRtpApp.vxe";
 char * argv[2];
 RTP_ID rtpId = NULL;
 /* set the application's arguments */
 argv[0] = vxeName;
 argv[1] = NULL;
 /* Spawn the RTP. No environment variables are passed. */
 if ((rtpId = rtpSpawn (vxeName, argv, NULL, 220, 0x10000, 0, 0)) == NULL)

  {
   printErr ("Impossible to start fooRtpApp.vxe (errno = %#x)", errno);
  }
}
```

20. Restart and reconnect the target as shown in steps 13 through 15.
21. Right-click the VIP and select **Rebuild Project**. Verify that your application launches automatically by observing the following output in the **Terminal** area.

    The following output confirms that the procedure operates properly:

```
-> I am main() in fooRtpApp
I am bar() in barLib
I fooRtpApp just called bar() in barLib
```

Note: If **LD_LIBRARY_PATH** environment variable not set, see step 15, the message **libc.so.1 not found** will pop-up on the target shell.

Previous topic: Connecting to the Target and Running the Example RTP Application on page 34

9. MANAGING PROJECT RESOURCES

1. Project Resource Management

Project resources include files, folders, and projects that you can create, import, and export.

For a description of project resource objects, see Project Resources on page 3.

Project resource management with Workbench 4 includes the following tasks:

- viewing resource properties
- importing and exporting resources
- adding new resources and removing resources
- moving resources from project to project
- creating linked resources
- comparing project resources

Related information

Viewing Resource Properties on page 39
Importing Resources into Projects on page 40
Exporting Resources from Projects on page 41
Adding New Resources to Projects on page 42
Removing Resources from Projects on page 43
Moving Resources from Project to Project on page 44
Creating Linked Resources on page 44
Comparing Project Resources on page 45

2. Viewing Resource Properties

You can view the properties of a Workbench resource.

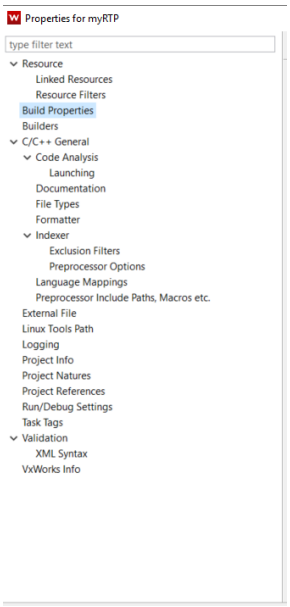
About This Task

To display the various properties of a Workbench resource, proceed as follows:

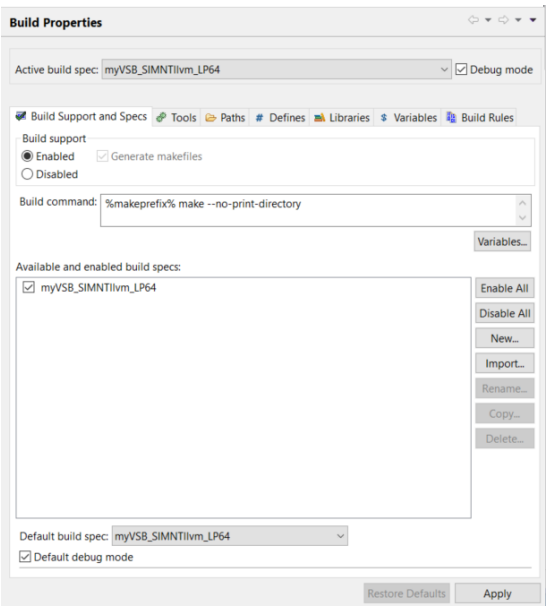
Procedure

1. Right-click a resource in one of the navigation views and select **Properties**.

WNDRVR



2. To view the build properties, select **Build Properties**.



Related information

Project Resource Management on page 39

3. Importing Resources into Projects

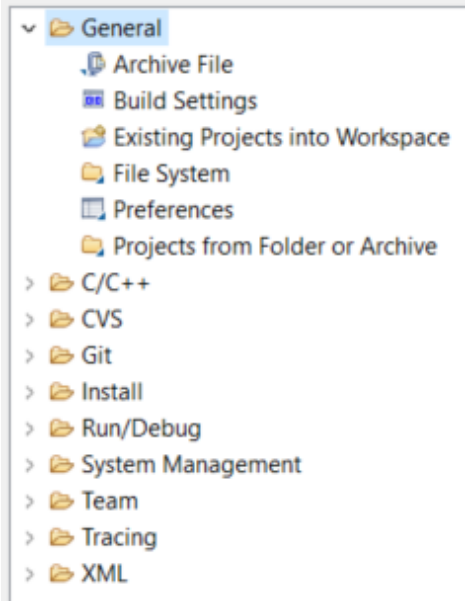
You can import existing resources into a project and import existing projects into your workspace.

About This Task

To import a resource, proceed as follows:

Procedure

1. Right-click a project in the Project Explorer and select **Import**.
2. From the Import dialog, select and import a resource.



To import files or folders from a file system, select **General > File System** and select the directories from which to import the resources.

You can also add existing files and folders to a project by dragging them to the project icon in the Project Explorer.

A third method of adding existing files and folders to a project is to copy and paste them into one of the navigation views.

To import an archive file, select **General > Archive File** and select the archive files you want to extract and import into the project.

To import an existing project into your workspace, select **General > Existing Project into Workspace** and select a project root directory or a project archive file to import.

Related information

Project Resource Management on page 39
Exporting Resources from Projects on page 41

4. Exporting Resources from Projects

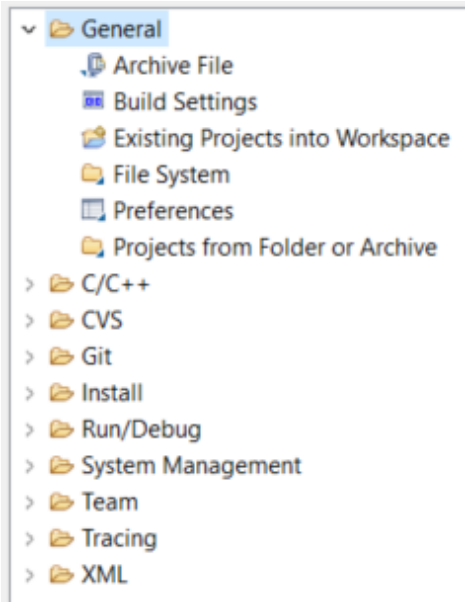
You can export project resources to a file system or an archive file.

About This Task

To export a resource from Workbench, proceed as follows:

Procedure

1. Select the resource or resources in the Project Explorer.
2. Right-click a selected resource and select **Export**.
3. From the Export dialog, select a destination and export the resources.



To export the resources to a file system, select **General > File System** and follow the instructions in the Export dialog.

You can also export files and folders from a project by dragging them to a file system.

A third method of exporting files and folders from a project is to copy and paste them into the file system.

To export resources to an archive file, select **General > Archive File** and follow the instructions in the Export dialog. By default, the resources that you selected are exported along with their children.

Related information

Project Resource Management on page 39
Importing Resources into Projects on page 40

5. Adding New Resources to Projects

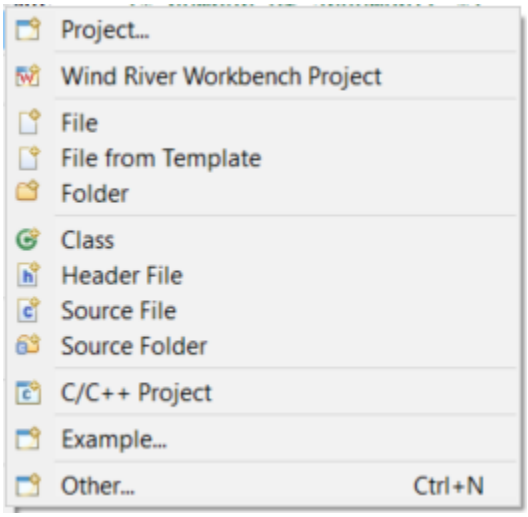
You can add new resources to a project including files, folders, and projects.

About This Task

To add a new resource into a project, proceed as follows:

Procedure

1. Access the Project Folder.
2. Right-click a project in the Project Explorer and select **New**.



3. Add new files, folders, and projects to a project using one of the following methods:
   - To add a new header file to the project, select **Header File** and enter the name of the file in the New Header File dialog.
   - To add a new source file to the project, select **Source File** and enter the name of the file in the New Source File dialog
   - To add a new folder to the project, select **Source Folder** and enter the name of the folder in the New Source Folder dialog.
   - To add a new project to an existing project, select **Project** and follow the instructions in the New Project dialog.

Related information

Project Resource Management on page 39
Removing Resources from Projects on page 43

6. Removing Resources from Projects

You can remove resources from projects by deleting the resource from the project.

About This Task

To remove a resource from a project, proceed as follows:

Procedure

1. Access the Project Folder.
2. Select one or more resources in the project and press the **Delete** key.

   You can also right-click a resource and select **Delete**.

Related information
Project Resource Management on page 39
Adding New Resources to Projects on page 42

7. Moving Resources from Project to Project

You can move resources from one project to another, or to another location.

About This Task

To move a resource from one project to another, proceed as follows:

Procedure

1. Access the Project Folder.
2. Select one or more resources in the project and drag the resources to another project.

   You can also right-click a selected resource and select **Move**, then select a project from the Move Resources dialog.

   Note: When moving a linked folder, the folder contents is not moved on the file system.

Related information
Project Resource Management on page 39

8. Creating Linked Resources

Folders and files can be linked to locations in a file system outside of the project's location.

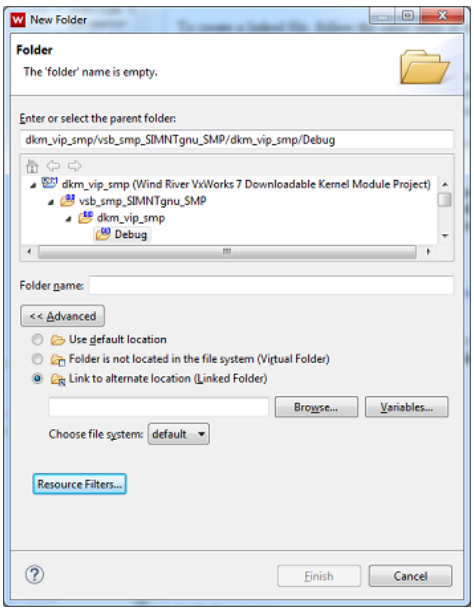
About This Task

For example, a linked folder can be used to store the build output separately from source files. Linked resource target paths can be either defined as absolute paths or relative to a path variable.

To create a linked resource, proceed as follows:

Procedure

1. Access the Project Folder.
2. Right-click a folder or project where you want a linked resource and select **New > Folder**.
3. From the New Folder dialog, specify the name of the parent folder or select a folder from the list.
4. Click **Advanced** and select **Link to alternate location (Linked Folder)**.



5. Enter a file system path or click **Browse** and select a folder in the file system.

Related information

Project Resource Management on page 39

9. Comparing Project Resources

You can compare project resources and display the differences in the file editors. The differences between resources are highlighted in the comparison editors.

About This Task

To compare files, proceed as follows:

Procedure

1. Select one or more resources in one of the navigation views, for example, the Project Explorer.
2. Right-click a selected resource, and select **Compare With**.
3. Select an option.

   Compare With > Each Other compares two or three selected resources with each other.

Compare With > Local History compares the selected resource to one that is in the local history, which is maintained when you save changes.

Related information

Project Resource Management on page 39